



Republic of Rwanda
Ministry of Education



RTB | RWANDA
TVET BOARD

GENFC401

FUNDAMENTALS OF C++ PROGRAMMING

Apply C++ Programming Fundamentals

Competence

RQF Level: 4

Learning Hours



Credits: 7

Sector: ICT & Multimedia

Trade: Computer System and Architecture

Module Type: General

Curriculum: ICTCSA4001TVET Certificate IV in Computer System and Architecture

Copyright: © Rwanda TVET Board, 2023

1200

Issue Date: September 2023

Purpose statement	This module Describes the skills, Knowledge and attitude required to Apply C++ Programming Fundamentals. This module intended to prepare students Pursuing TVET Level 4 in Computer System and Architecture. At the end of this module, the students will be able To apply basic C++ concepts, Apply OOP concepts, Perform CPU Optimization.					
Learning assumed to be in place	• C Programming					
Delivery modality	Training delivery		100%	Assessment		Total 100%
	Theoretical content		20%	Formative assessment	20%	50%
	Practical work:		80%		80%	
	Group project and presentation	30%				
	Individual project /Work	50%				
			Summative Assessment		50%	

Elements of Competence and Performance Criteria

Elements of competence	Performance criteria
1. Apply basics C++ Concepts	1.1 Development Environment is prepared properly based on C++ programming standards.
	1.2. Variables are properly applied based on programming rules and guidelines
	1.3. Operators are properly applied based on programming rules and semantics defined
	1.4. Control structure are properly applied based on the specific rules and syntax defined
	1.5. Functions are correctly applied based on task to be done
	1.6. Arrays are properly applied based on programming rules and conventions
2. Apply OOP Concepts	2.1. Class and object are properly applied based on the principles of object-oriented programming.
	2.2. Inheritance and polymorphism are properly applied based on the rules and principles of object-oriented programming.
	2.3. Namespaces are properly applied based on the rules and conventions
	2.4. Errors and Exceptions are effectively handled according to the rules and practices of error handling

3. Perform CPU Optimization	3.1. Pointers are properly applied based on programming rules and specific concepts of pointer manipulations
	3.2. File handling are properly applied based on the rules and concepts of file Input/ Output
	3.3. Multithreading and concurrency are effectively applied based on the computer performance needed.
	3.4 Inline-assembly is appropriately applied based on processor control of hardware architecture.

Intended knowledge, skills and attitude

knowledge	Skills	Attitude
✓ Define basic principles of OOP ✓ Describe IDE ✓ Identify various diagnostic tools	✓ Create a C++ programs ✓ Compile a C++ programs ✓ Test a C++ programs ✓ Debug a C++ programs ✓ Execute a C++ programs ✓ Backup and recovery	✓ Creative ✓ Self-driven ✓ Collaborative

Course content

Learning outcomes	At the end of the module the learner will be able to: 1. Apply basic C++ Concepts 2. Apply OOP Concepts 3. Perform CPU Optimization
--------------------------	--

Learning outcome 1: Apply basic c++ Concepts	Learning hours: 20
Indicative content	
● Preparation of Development Environment ✓ Introduction to programming  Definition  Extension  Features  Applications ✓ Description of C++ Tools  Ide  Text Editor  Compiler and Interpreter  Debugging  Version Control ✓ Tools installation  IDE Installation  Compiler Installation  Setup Environment Variable Path	

✓ Test Development environment

 Description of IDE menus and Icons

 Description of C++ syntax

✓ Execute default program

- **Apply variables**

✓ Definition

✓ Identification of Reserved Keywords

✓ Description of Data Types

✓ Declaration of Variable

 Identifier

 Naming Rules

 Scope of Variable

 Initialization

- **Apply of Operators**

✓ Description of operator

 Definition

 Precedence and Associativity

 Operator overloading

✓ Types of operator

- **Apply control structure**

✓ Selection

 Conditional Statement

 Switch Case Statement

✓ Iteration

 Loops

 Nested Loops

✓ Jump statement

+ Break

+ Continue

+ Goto

+ Return

● **Apply functions**

✓ Description of function

+ Purpose

+ Function Declaration And Definition

+ Function Call

+ Function Parameters

+ Default Argument

✓ Types of Functions

+ Built-In Function

+ User Defined Function

+ Recursive Function

+ Inline Functions

+ Lambda Functions

+ Prototyping Function

● **Apply Array**

✓ Descriptions

+ Array Definition

+ Array Declaration

+ Array Initialization

+ Accessing Array Element



Iterating Over Array

✓ Types of arrays



One-Dimensional array



Two-Dimensional array



Multidimensional array



Dynamic array

Resources required for the learning outcome

Equipment	- Computer - Internet Connection - Storage device
Materials	- Book - Online Tutorials
Tools	- C++ Compiler - Integrated Development Environment (Ide) - Text Editors
Facilitation techniques	- Demonstration - Individual and group work - Trainer guided
Formative assessment methods /(CAT)	- Written assessment - practical assessment

Learning outcome 2. Apply OOP Concepts	Learning hours: 30
Indicative content	
Apply Class and object	

- ✓ Description of key terms
 -  Class
 -  Object
 -  Encapsulation
 -  Data Abstraction
 -  Constructors And Destructors
 -  Inheritance
 -  Polymorphism
 -  Function Overloading
 -  Function Overriding
- ✓ Advantages of using OOP
- ✓ Way of Creating Objects
- ✓ Access Specifiers
- ✓ Constructors And Destructors
 -  Copy Constructor
 -  Deep Copy
 -  Shallow Copy
 -  Destructor
 -  Constructor Overloading
- ✓ Static
 -  Member Variable
 -  Member Function
- ✓ Constants
 -  Constant Member Variable
 -  Constant Functions

- **Apply Inheritance and polymorphism**

- ✓ Inheritance

- ✚ Syntax

- ✚ Relationship Between Base And Derived Classes

- ✚ Create A Derived Class from A Base Class.

- ✚ Adding A Property to An Object

- ✚ Adding A Method to An Object

- ✚ Friend Classes and Method

- ✓ Polymorphism

- ✚ Pointers and References

- ✚ Function overloading

- ✚ Function Overriding

- ✚ Virtual Functions

- ✓ Implement abstract class

- ✓ Implement interface

- ✓ Object

- ✚ Composition

- ✚ Aggregation

- ✚ Nesting

- ✚ Serialization

- ✚ Deserialization

- **Apply Namespaces**

- ✓ Description of Namespace

- ✚ Definition

- ✚ Syntax



Mechanism



Rules

✓ Creating Namespaces

✓ Using Namespaces



Accessing elements



Importing



Nested namespaces



Alias



Standard library

✓ Handling Naming Conflicts

- **Errors and Exceptions handling**

✓ Types of Errors

✓ Exception Handling Mechanism



The try block



The throw statement



The catch block

✓ Exception Safety and Resource Management techniques



No-throw guarantee



Basic guarantee



No guarantee

Resources required for the learning outcome

Equipment

- Computer
- Internet Connection
- Storage device

Materials

- Book

	▪ Online Tutorials
Tools	▪ C++ Compiler ▪ Integrated Development Environment (IDE) ▪ Text Editors
Facilitation techniques	▪ Demonstration ▪ Individual and group work ▪ Trainer guided
Formative assessment methods /(CAT)	▪ Written assessment ▪ Practical Assessment

Learning outcome 3: Perform CPU Optimization	Learning hours: 20
Indicative content	
● Apply pointers ✓ Description of pointers + Definition + Memory addresses + Relationship between variables and pointers. ✓ Declaration and Initialization + Syntax + Pointer data types + Ways to initialize pointers ✓ Dereferencing pointers ✓ Pointer arithmetic ✓ Passing pointers as function argument ✓ Use of Dynamic Memory Allocation	

- + New operators

- + Delete operators

- + Handling null

- + Memory leak

- + Smart pointer

- ✓ Array of pointers

- ✓ array of function pointers

- **Apply file handling**

- ✓ Introduction to File Handling

- + Explain The Concept

- + Importance In C++ Programming.

- + Benefits Of File I/O

- ✓ Header File and Stream Classes

- + Introduce the <fstream> header file and its role in file handling

- + Mainstream classes

- ✓ Modes

- + Open

- + Reading

- + Writing

- + Append

- + Truncate

- + Binary

- + Close

- **Apply multithreading and concurrency**

- ✓ Create and manage threads using std::thread Class

- ✓ Apply Thread Synchronization primitives



Std::mutex



Std::lock_guard



Std::unique_lock

✓ Apply Deadlocks and Race Conditions

✓ Implement Thread Pool

✓ Apply Parallel Algorithms



std::for_each



std::transform



std::reduce

✓ Implement Parallel Data Processing

✓ Implement Task-Based Parallelism

✓ Implement Load Balancing

● **Apply inline assembly**

✓ Description of Inline assembly



Assembly language



Asm



Fine-grained



Machine code



Syntax

✓ I/o operands

✓ Use register constraints

✓ Low level optimization

✓ Hardware interfacing



I/O Ports



Access to Processor Registers

Resources required for the learning outcome

Equipment	▪ Computer ▪ Internet Connection ▪ Storage device
Materials	▪ Book ▪ Online Tutorials
Tools	▪ C++ Compiler ▪ Integrated Development Environment (IDE) ▪ Text Editors
Facilitation techniques	▪ Demonstration ▪ Individual and group work ▪ Trainer guided
Formative assessment methods /(CAT)	▪ Written assessment ▪ Practical assessment

References

1. Udemy. (n.d.). *C Programming Language: Master the C Language from its Fundamentals to Advanced*. Retrieved from Udemy website: <https://www.udemy.com/clang-algo-ds/>
2. Udemy. (n.d.). *Data Structures and Algorithms in C++ for Coding Interview*. Retrieved from Udemy website: <https://www.udemy.com/data-structures-and-algorithms-in-c/>
3. edX. (n.d.). *Introduction to JavaScript*. Retrieved from edX website: <https://www.edx.org/course/c++-introduction/>
4. Stroustrup, B. (2013). *The C++ Programming Language (4th ed.)*. Addison-Wesley Professional.
5. Prata, S. (2017). *C++ Primer Plus (6th ed.)*. Addison-Wesley Professional.
6. Lippman, S., Lajoie, J., & Moo, B. (2012). *C++ Primer (5th ed.)*. Addison-Wesley Professional.

7. *Eckel, B. (2018). Thinking in C++, Volume 1: Introduction to Standard C++ (2nd ed.). MindView Press.*
8. *Deitel, P. J., & Deitel, H. M. (2017). C++ How to Program (10th ed.)*